Traffic Volume Based Routing Guidance Web App For Boroondara Area

Machine learning-based traffic prediction and intelligent route planning algorithms.

Team member:
Khoi Nguyen Pham
Minh An Nguyen
Thanh Nam Vu

Contents

1 Introduction **2**

2 Instructions **2**
2.1 Setup Requirements . . . 2
2.2 Using the Application . . . 2

3 Features/Bugs/Missing **3**
3.1 Features . . . 3
3.2 Bugs and Missing . . . 4

4 Testing **5**

5 Insights **5**
5.1 Introduction to the Insights Section . . . 5
5.2 Overview of Algorithms . . . 5
5.3 Precision Metrics & Model Generalization . . . 6
5.4 Convergence Rate Analysis . . . 8

6 Research **10**
6.1 Streamlit Web App for Interactive Route Guidance . . . 10
6.2 Research Models to Find the Best Fit Model . . . 10

7 Conclusion **12**

References **13**

1 Introduction

The **Traffic Prediction** and **Traffic-based Route Guidance System** addresses two critical challenges in urban transportation: accurately predicting traffic conditions and finding optimal routes between locations.

This project implements a comprehensive solution combining machine learning-based traffic prediction and intelligent route planning algorithms. We have implemented several **deep learning models** for traffic prediction, including **LSTM**, **GRU**, and **Bi-LSTM** architectures. These models are trained on historical traffic data to predict traffic volumes at different locations and times.

The system uses location embeddings and attention mechanisms to capture spatial and temporal traffic flow patterns. For route finding, we have implemented multiple **search algorithms**, including **A***, **Breadth-First Search (BFS)**, **Depth-First Search (DFS)**, **Greedy Best-First Search (GBFS)**, **Uniform Cost Search (UCS)**, and **Fringe Search**. Each algorithm offers different trade-offs between optimality, completeness, and computational efficiency.

The route-finding process considers the predicted traffic conditions to calculate travel times and suggest optimal routes. The system provides an interactive web interface where users can:

- Select their origin and destination
- Choose their preferred prediction model and search algorithm
- View detailed route information including travel times, traffic conditions, and intermediate stops

The routes are displayed on an interactive map with color-coded traffic conditions for easy visualization.

2 Instructions

2.1 Setup Requirements

To let the program run smoothly, first, the user needs to install the **dependencies**:

- `pip install -r requirements.txt`

After installing all the dependencies, run the script: `"streamlit run app.py"`, and the application will automatically run in the default web browser using the local host. Alternatively, we've also hosted the program on Streamlit and the users can the website access at: `https://traffic-based-route-guidance-system.streamlit.app`.

2.2 Using the Application

Navigation

The application has two main pages accessible from the **sidebar**:

1. **Network Map**: View the entire traffic monitoring network
2. **Route Finder**: Find optimal routes between sites

Finding Routes

1. **Select Origin and Destination**
   - Choose your starting point (**Origin**) from the dropdown menu
   - Choose your **destination** from the dropdown menu
   - Each selection will show details about the selected site
2. **Configure Route Options**
   - **Algorithms**: Choose one or more search algorithms:

  - **A*** (A-Star Search)
  - **DFS** (Depth-First Search)
  - **BFS** (Breadth-First Search)
  - **GBFS** (Greedy Best-First Search)
  - **UCS** (Uniform Cost Search)
  - **Fringe Search**
- Select `"All"` to compare all algorithms
- **Prediction Model**: Choose a traffic prediction model:
  - **LSTM** (default)
  - **GRU**
  - **BiLSTM**

3. **Set Date and Time**
   - Select a date between **October 1, 2006**, and **November 30, 2006**
   - Choose the **hour (0–23)** and **minute**
   - **Note**: Times are automatically rounded to the nearest 15-minute interval
4. **Find Routes**
   - Click the **"Find Routes"** button
   - The system will calculate **optimal routes** based on your selections

3 Features/Bugs/Missing

3.1 Features

Data Processing and Integration

The project began with robust processing of the raw SCATS traffic dataset [1], which contains 15-minute interval data for each monitored intersection across October 2006. Using Python's pandas library, we loaded and validated the CSV data, ensuring correct timestamp parsing and column integrity. The original data was reshaped from wide format (96 columns per day) to long format, producing a more structured and usable representation for time-series modeling.

Each location's data was then cleaned by removing duplicates and sites with insufficient records. Additional preprocessing involved analyzing spatial relationships between SCATS numbers and their approach directions. All problematic or incomplete data points were excluded to maintain dataset quality. The cleaned data was saved and used as input for the feature engineering pipeline.

Feature Engineering

To prepare the SCATS data for time-series prediction, we built a special feature engineering pipeline. First, we created time-related features. This included day-of-week indicators using sine and cosine functions, written as `dow_sin` and `dow_cos`. We also encoded the time of day using `tod_sin` and `tod_cos`. These sinusoidal features help the model understand repeating patterns.

Next, we added binary flags. For example, `is_weekend` shows whether the data is from a weekend. The `after_gap` flag shows if there is a break in the data sequence. To detect gaps, we calculated `days_since_prev` for each site. This stops the model from learning from broken or incomplete time series.

We also created lag features. These give the model access to traffic values from the past. We used lags at 15 minutes (`lag_1`), 1 hour (`lag_4`), and 1 day (`lag_96`). A baseline feature, `avg_traffic_this_timeofday`, was added to show typical traffic for each site and time.

Each location was turned into a numeric code using `location_idx`. This is useful for embedding layers in the model.

After building the full set of features, we created supervised learning sequences. These were split into fixed lengths, like 24 time steps. We used `StandardScaler` to normalize the data. Finally, we split the dataset into training and testing sets based on date. This keeps the training data in the past and the testing data in the future, which is important for time-series forecasting.

Machine Learning Models

Five deep learning models were developed: LSTM, GRU, Bidirectional LSTM (BiLSTM), CNN-BiLSTM, and CNN-BiGRU. The first three models (LSTM, GRU, and BiLSTM) were used for traffic flow prediction. The CNN-based models were created for additional research purposes. All models were designed to handle time-series input and included location embeddings to capture traffic behavior differences across SCATS sites.

- **LSTM**: A unidirectional Long Short-Term Memory model that captures sequential patterns over time.
- **GRU**: A more lightweight Gated Recurrent Unit model that maintains performance with fewer parameters.
- **BiLSTM**: Our custom approach, extending LSTM with bidirectional context to capture both past and future trends within a time window.
- **CNN-BiLSTM**: A hybrid model combining 1D convolutional layers for short-term pattern extraction with a BiLSTM backend for sequential learning. This model will be further explored as part of our research component.
- **CNN-BiGRU**: A similar hybrid structure combining convolutional preprocessing with a BiGRU model. This model will be further explored as part of our research component.

Each model was integrated into a unified training pipeline using Keras and TensorFlow. This pipeline supports configurable model parameters, early stopping, adaptive learning rate scheduling, and automatic checkpointing. Post-training, the models were evaluated on a holdout test set using RMSE, MAE, and R^2 metrics to enable fair and consistent performance comparisons.

Travel Time Estimation

To enable realistic travel time predictions, we implemented a conversion formula provided by the course to translate traffic volumes into estimated travel times. This involved accounting for the volume at each SCATS site and the geographic distance between locations. An average delay of 30 seconds was also added at each intersection to reflect real-world traffic signal delays. These travel time estimates were later used to weight edges in the route-finding graph for optimal path selection.

Route Finding and Network Integration

We extended the search capabilities from Assignment 2A by integrating six different search algorithms: A*, DFS, BFS, UCS, GBFS, and Fringe Search. Each intersection was treated as a node, and travel time served as the edge cost. The heuristics for informed search algorithms are now adjusted to use the Haversine distance between intersections to guide the search efficiently. The road network graph was dynamically built using SCATS site relationships and geographic coordinates.

To support efficient routing, we used the three trained models: LSTM, GRU, and BiLSTM, to predict traffic volumes for all SCATS locations covering the period from October to November 2006. These predictions were stored in CSV files. When the user selects a specific model and time for route planning, the system retrieves the corresponding predicted traffic volumes from these files and then converts them into travel speeds and travel times in real time. This design enables dynamic, model-driven routing while maintaining fast response times.

User Interface and Visualization

A Streamlit-based web interface was developed to provide interactive access to all core functionalities. Users can specify origin, destination, preferred algorithm, and prediction model. The system then visualizes routes on a map with traffic-aware coloring and provides detailed breakdowns of estimated travel time and distance. This component will be further refined and expanded as part of our research initiative on predictive route visualisation.

3.2 Bugs and Missing

No bugs or missing issues were identified during the development and testing of the program.

4 Testing

To evaluate the correctness, efficiency, and robustness of the TBRGS system, ten targeted test cases were developed. Each test focuses on specific aspects of the system to ensure overall functionality and to assess its performance under varied conditions.

ID	Description	Test Setup	Expected Results	Status
1	Isolated Sites Test	Origin: Site with no connections (e.g., isolated site) Destination: Any other site	System shows error message indicating no routes found	Pass
2	Date Range Restriction	Attempt to select dates outside Oct-Nov 2006	UI prevents selection or shows appropriate error message	Pass
3	Traffic Flow to Travel Time Conversion	Origin-Destination with known traffic volumes	Travel times match expected values based on formula	Pass
4	Algorithm Comparison	Origin: 2000 (WARRIGAL_RD/TOORAK_RD) Destination: 3002 (DENMARK_ST/BARKERS_RD) Test each algorithm separately	Each algorithm finds valid routes with appropriate optimization	Pass
5	Time Progression Handling	Long route (5+ segments) at rush hour (8:00 AM) vs. non-rush hour (11:00 AM)	Travel time estimates for later segments reflect time-of-day changes in traffic	Pass
6	Multiple Origin-Destination Testing	Test multiple different O-D pairs across the network to ensure consistent routing	System successfully generates routes for various geographic combinations	Pass
7	ML Model Comparison	Same O-D pair tested with all three models (LSTM, GRU, Bi-LSTM)	Different models show measurable variations in predicted routes	Pass
8	Long Distance Routes	Select origin and destination at opposite ends of map	System finds efficient routes through multiple connections	Pass
9	Road Network Topology Validation	Test routes between sites known to be connected by specific roads (e.g., WARRIGAL_RD between consecutive intersections)	Routes follow the actual road network topology, with connections matching the physical street layout of Boroondara	Pass
10	Response Time	Complex route calculation with multiple algorithms	Routes calculated and displayed within 10 seconds	Pass

Table 1: TBRGS System Test Cases

5 Insights

5.1 Introduction to the Insights Section

This section thoroughly examines three neural network architectures, which are LSTM, GRU, and BiLSTM, implemented in our Traffic-Based Route Guidance System. In this section, we systematically evaluate these models using MAE, RMSE, R^2, validation loss, and convergence behavior to assess both training efficiency and predictive performance.

5.2 Overview of Algorithms

The task provides 2 model to start with, which are LSTM and GRU. Moreover, BILSTM was chosen as our custom model. Brief technical description of each:

- **LSTM (Long Short-Term Memory)**: Utilizes a persistent cell state regulated by three specialized gates, which are forget, input, and output gates. The forget gate determines which information to discard, the input gate controls new information addition, and the output gate filters the final output. Therefore, this architecture effectively preserves long-term dependencies across extended traffic sequences while addressing the vanishing gradient problem appearing in the old RNN (Recurrent Neural Network) [2] architecture through its controlled memory mechanisms. [3]
- **GRU (Gated Recurrent Unit)**: Implements a more efficient design with only two gates: the update and reset gates. The update gate combines LSTM's forget and input functionality, while the reset gate controls previous state influence on the current candidate state. This streamlined structure reduces computational requirements by approximately 15% compared to LSTM, accelerating training without significantly sacrificing predictive capabilities for traffic flow patterns. [4]
- **BiLSTM (Bidirectional LSTM)**: Deploys two parallel LSTM layers processing traffic sequences in opposite directions, which are forward and backward. By simultaneously analyzing past-to-future and future-to-past information flows, BiLSTM captures comprehensive contextual information at each timestep. As a result, this bidirectional processing especially benefits traffic prediction by recognizing how congestion propagates both upstream and downstream through interconnected road networks, providing more nuanced predictions at network bottlenecks and intersections. [5]

5.3 Precision Metrics & Model Generalization

Precision Metrics

Employing R², RMSE, and MAE together provides a comprehensive model assessment for traffic prediction. R² measures how well models capture underlying traffic pattern variability, with our high values (more than 0.95) confirming effective pattern recognition regardless of error magnitude. RMSE disproportionately penalizes large prediction errors, revealing performance during critical peak congestion periods where accuracy matters most. MAE offers the most intuitive interpretation, which is simply the average number of vehicles by which predictions deviate from reality, giving a clear understanding of expected day-to-day accuracy[6][7]. Together, these complementary metrics evaluate typical performance, critical event handling, and pattern recognition capability, ensuring our selected model delivers reliable predictions across diverse traffic scenarios.
After training all 3 models, we have successfully conducted a metric performance comparison of the models table below:

Model	MAE	RMSE	R^2
LSTM	13.1577	18.8788	0.9521
GRU	13.5098	18.6302	0.9534
BiLSTM	12.6369	18.4175	0.9544

Table 2: Performance comparison of three models

The BiLSTM model consistently outperforms both LSTM and GRU architectures across all evaluation metrics. With the lowest Mean Absolute Error (12.6369) and Root Mean Square Error (18.4175), BiLSTM demonstrates superior prediction accuracy, particularly for extreme traffic conditions.

Furthermore, its slightly higher coefficient of determination ($R^2 = 0.9544$) indicates that the bidirectional approach more effectively captures the underlying patterns in traffic flow variation. GRU performs marginally better than LSTM in terms of RMSE and R^2, despite having a slightly higher MAE. This suggests that while GRU may have more outlier predictions (affecting MAE), its overall prediction distribution more closely aligns with the actual traffic patterns. The comparable performance between GRU and LSTM is particularly noteworthy considering GRU's reduced computational complexity. With the superior final metrics, it strongly supports our hypothesis that bidirectional processing of BILSTM provides meaningful advantages for traffic prediction tasks by incorporating both upstream and downstream traffic influences.

Model Generalization

During the training process, three line plots were created illustrating the training and validation loss curves for LSTM, GRU, and BiLSTM models to test each model's generalization.

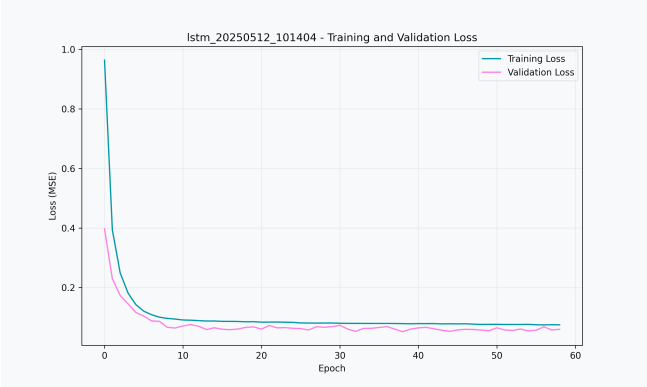

Figure 1: LSTM model Training and Validation Loss visualization by Matplotlib

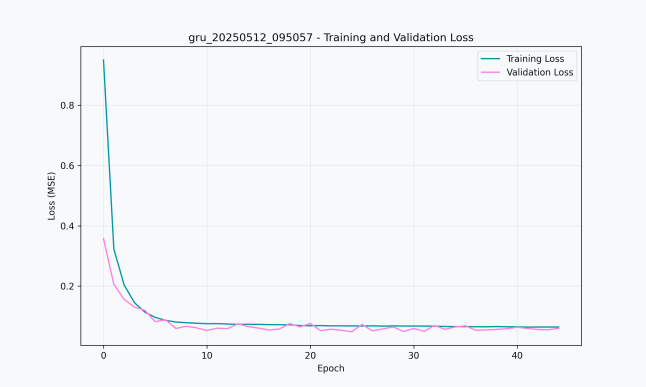

Figure 2: GRU model Training and Validation Loss visualization by Matplotlib

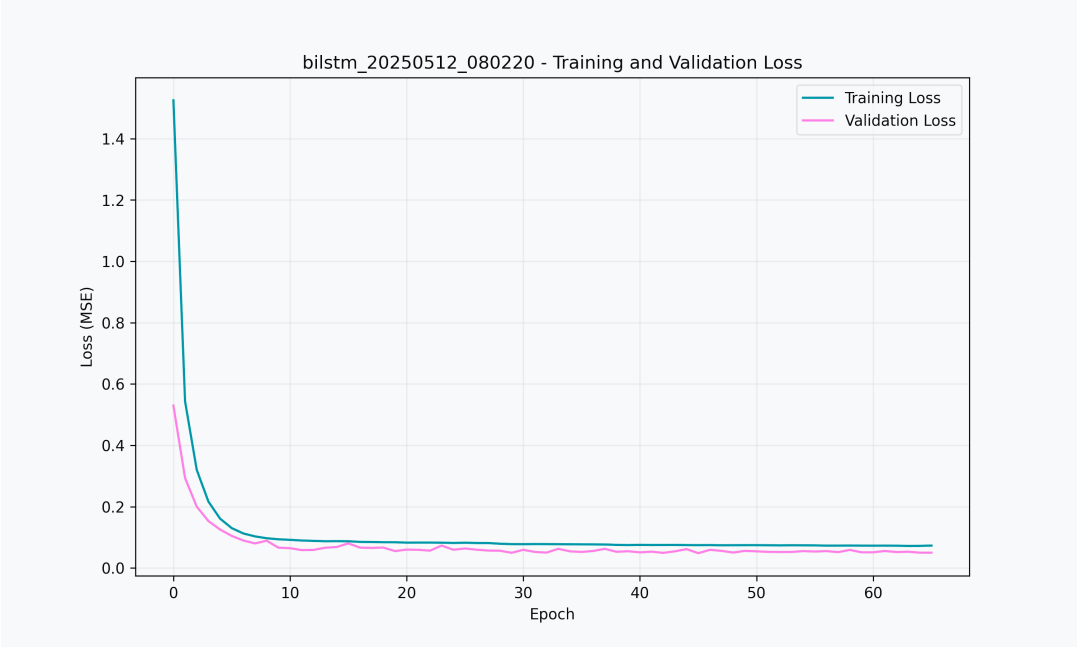

Figure 3: BiLSTM model Training and Validation Loss visualization by Matplotlib

All three architectures demonstrate excellent convergence patterns, with both training (blue) and validation (orange) loss decreasing rapidly in the initial epochs and then stabilizing, maintaining a small and consistent gap between them throughout the later epochs, indicating good model fitting without significant overfitting or underfitting. This convergence behavior directly correlates with the performance metrics shown in the table, where BiLSTM achieves the best results across all evaluation metrics (MAE: 12.6369, RMSE: 18.4175, R^2: 0.9544). As depicted in the plots, the BiLSTM model (Figure 8) converges to the lowest final loss values (approximately 0.05), confirming its superior capabilities in capturing bidirectional dependencies in the data. The comparable convergence patterns of GRU and LSTM align with their similar performance metrics, though GRU shows slightly faster initial convergence, consistent with its computational efficiency advantages mentioned in the strengths and weaknesses section.

Model Performance

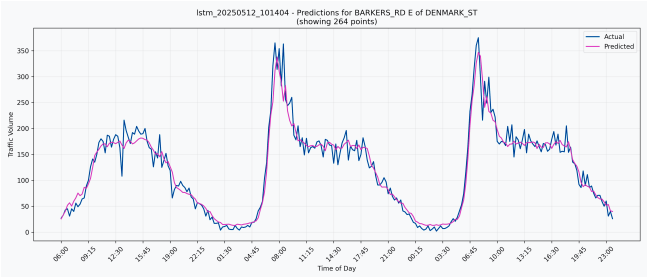

Figure 4: LSTM model Performance on BARKERS_RD_E_of_DENMARK_ST by Matplotlib

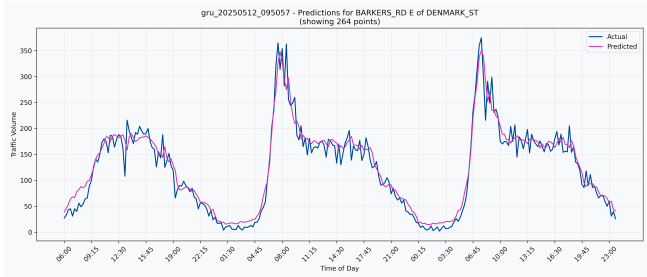

Figure 5: GRU model Performance on BARKERS_RD_E_of_DENMARK_ST by Matplotlib

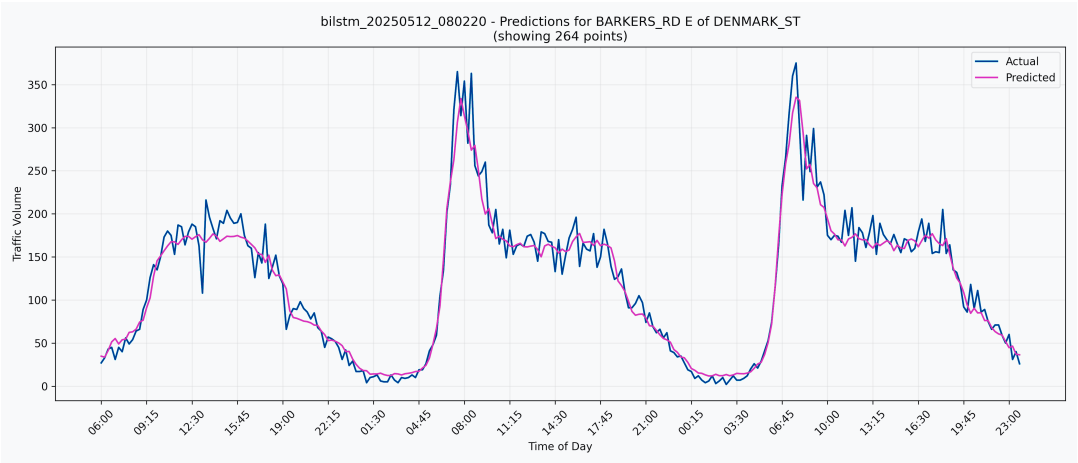

Figure 6: BiLSTM model Performance on BARKERS_RD_E_of_DENMARK_ST by Matplotlib

The predictive performance of LSTM, GRU, and BiLSTM models for traffic volume predictions on BARKERS_RD E of DENMARK_ST is illustrated in Figures 2, 3, and 4. All three models demonstrate strong capabilities in capturing the overall morning traffic patterns, with particularly accurate predictions during the morning (06:00-09:00) and afternoon (15:00-18:00) peak periods. The BiLSTM model performs marginally better at tracking the sharp transitions during peak traffic hours, especially visible around 08:00 and 03:30, where the actual traffic values reach approximately 360 units. The GRU model shows comparable performance to BiLSTM but demonstrates slightly less precision in capturing the highest amplitude peaks, particularly around the morning rush hour (08:00). The standard LSTM model, while effective at modeling the general traffic flow patterns, appears to slightly underestimate the magnitude of sudden fluctuations during high-volume periods. All three models effectively capture the overnight low-traffic periods (22:00-04:00) with minimal deviation from actual values. The predictions also accurately reflect the three distinct traffic waves throughout the day, with the models performing consistently across multiple days of data (264 data points).

Overall, BiLSTM offers marginal improvements in prediction accuracy for rapid traffic transitions, likely due to its bidirectional architecture processing temporal information in both forward and backward directions, though the performance differences between the three approaches are subtle for this particular traffic prediction task.

5.4 Convergence Rate Analysis

To evaluate training efficiency, we compare how quickly the three models, which are GRU, LSTM, and BiLSTM, converged to a stable low loss. The `epoch_loss` plot shows how the training loss decreased over time for each model.

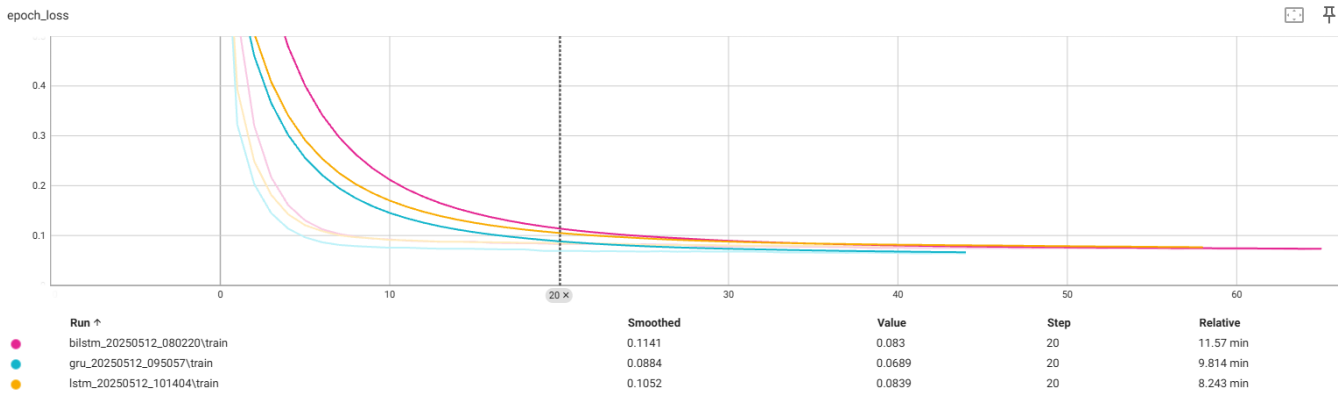

Figure 7: Epoch loss of 3 models after each 20 steps visualization by TensorBoard.

- **GRU** displayed the steepest initial loss decline within the first 10 epochs, indicating it learned the fastest. It achieved a smoothed loss of **0.0884** in **20 steps**.
- **LSTM** showed the second-fastest convergence, reaching a smoothed loss of **0.1052** in **20 steps**, which is slower than GRU.
- **BiLSTM** converged the slowest, with a smoothed loss of **0.1141** in **20 steps**.

This ranking aligns with architectural expectations: GRU, with fewer gates and a simpler structure, learns faster than LSTM; BiLSTM, due to its dual-pass nature, is the slowest to converge because its bidirectional architecture (processing input both forward and backward) increases computational complexity and training time. In conclusion, these findings highlight the trade-off between learning speed and potential accuracy gains from more complex architectures.

Strengths and Weaknesses of Each Algorithm

LSTM

- **Strengths:** Solid baseline performance with decent generalization capabilities and effective memory retention for traffic pattern recognition.
- **Weaknesses:** Slower convergence compared to GRU because it utilizes three gates (input, forget, and output), resulting in more computational time consumed and higher complexity. This architecture, while thorough, requires more resources and training iterations to reach optimal performance. [8]

GRU

- **Strengths:** Different from LSTM, GRU (introduced in 2014 after LSTM) features an update gate (combining input gate and forget gate from LSTM) that enables fastest runtime and convergence speed among the tested models. Its streamlined architecture with just two gates significantly reduces computational demands.
- **Weaknesses:** Despite its efficiency, GRU showed worse generalization and had the highest Mean Absolute Error (13.5098) among the tested models. The validation loss also exhibited slightly more fluctuation, indicating less stability during training compared to other architectures. [8]

BiLSTM

- **Strengths:** BiLSTM achieved the best loss performance overall, produced fewer large prediction errors, and demonstrated the most stable training behavior. Its bidirectional approach effectively captured complex traffic interdependencies, leading to superior metrics across MAE (12.6369), RMSE (18.4175), and R^2 (0.9544).

- **Weaknesses:** Due to its implementation of two parallel LSTM layers processing traffic sequences in opposite directions, it has the most complex architecture among the models tested, resulting in higher computational costs and longer training times. This increased complexity, while beneficial for prediction quality, presents challenges for deployment in resource-constrained environments. [9]

6 Research

6.1 Streamlit Web App for Interactive Route Guidance

As part of our research component, we developed an interactive web-based interface for the TBRGS system using `Streamlit` for the frontend and `Folium` for map visualization. This web app allows users to explore the SCATS network, compare multiple route options, and visualize traffic-aware paths based on real-time prediction data.

On the backend, the system builds upon the `SiteNetwork` class, which processes SCATS metadata and constructs a directed graph of site-to-site connections based on road alignment and approach direction. To ensure the intersections align accurately with real-world locations on the map, a manual adjustment was applied to the latitude and longitude coordinates of each SCATS site. The `RouteFinder` module dynamically loads precomputed traffic volume predictions from CSV files generated by our LSTM, GRU, and BiLSTM models. These volumes are converted into travel times using a quadratic speed formula, enabling realistic, time-sensitive routing. When a user specifies a time (e.g., 08:27), the system automatically rounds it down to the nearest 15-minute interval (e.g., 08:15) to align with the prediction intervals.

The frontend comprises modular `Streamlit` pages. The `NetworkPage` allows users to interactively browse the SCATS network. A user can search for a specific SCATS site, and the system will highlight that site on the map, display the roads it connects to, and list its incoming and outgoing connections to other sites as derived from the connection dataset. This makes it easy to visually inspect local connectivity and traffic topology.

The `RoutePage` allows users to select an origin, destination, date, time, ML model, and routing algorithm(s). It then invokes the backend route finder and displays the results, including ranked route summaries, detailed step-by-step breakdowns, and interactive maps with traffic-level color coding. All maps are rendered using `Folium`, including dynamic polylines and site markers enhanced with tooltips and popups.

Up to six routes, each generated by a different search algorithm (A*, BFS, DFS, UCS, GBFS, Fringe), are returned for comparison. These are color-coded from best to worst (e.g., green for the fastest and black for slower alternatives), enabling users to visually assess route quality. This interface not only improves system accessibility but also supports experimentation with different models and algorithms. It provides clear visual feedback on how traffic patterns and search strategies affect route selection.

Here's the LaTeX code for your section, properly formatted with subsections and paragraphs. I've added appropriate spacing and formatting to enhance readability in a LaTeX document:

6.2 Research Models to Find the Best Fit Model

Overview of Two Research Models

With the aim of finding the most suitable model for our Traffic-Based Route Guidance System (TBRGS), and building on previous work with LSTM, GRU, and BiLSTM models, we conducted a study on two additional hybrid architectures **CNN-BiLSTM** and **CNN-BiGRU**.

- **CNN-BiLSTM**: is a powerful hybrid architecture that combines **spatial pattern recognition** with **bidirectional temporal memory**, making it well-suited for tasks such as traffic prediction, anomaly detection, and time-series forecasting where both spatial and temporal patterns are crucial. The CNN component captures how congestion and other patterns form **spatially** across a road network, while the BiLSTM component processes these features in both **forward and backward time directions**, enabling the model to consider both **past trends** and **upcoming conditions** when forecasting future traffic states. This architecture often achieves better **accuracy** and more **stable learning**, especially in complex or long-range sequence modeling tasks.

- **CNN-BiGRU**: Combines **pattern recognition** with **bidirectional sequence processing** for traffic prediction. The CNN component identifies **spatial traffic patterns** like rush hour trends and congestion formations, while the BiGRU processes these patterns in both forward and backward time directions using an efficient **two-gate structure**. This hybrid approach captures how traffic conditions spread both **upstream and downstream** through road networks while requiring less **computational power** than LSTM-based alternatives. CNN-BiGRU achieves strong **predictive performance** through its ability to identify important traffic patterns and understand their evolution over time in both directions.

Model Generalization & Regression Metrics Comparison

To comprehensively evaluate the predictive performance of both models on unseen data, we used standard regression metrics that provide quantitative measurements of prediction accuracy after the training process.

Model	MAE	RMSE	R^2
CNN-BiGRU	16.8298	11.2363	0.9620
CNN-BiLSTM	16.8748	11.2470	0.9617

Table 3: Regression metrics for CNN-BiGRU and CNN-BiLSTM models

The regression metrics reveal that both models perform remarkably similarly on unseen data. CNN-BiGRU demonstrates marginally better predictive performance across all three metrics, with a slightly lower Mean Absolute Error (MAE) of 16.8298 versus 16.8748 for CNN-BiLSTM (a difference of approximately 0.3%). Similarly, Root Mean Square Error (RMSE) shows a minimal difference of 0.0107 in favor of CNN-BiGRU, while the coefficient of determination (R^2) values are virtually identical at 0.962 and 0.9617, respectively. Both models demonstrated strong learning performance, achieving an epoch loss between **0.0534** and **0.0535**. The next step is to evaluate their ability to generalize to unseen data by comparing **training loss** and **validation loss** for each model.

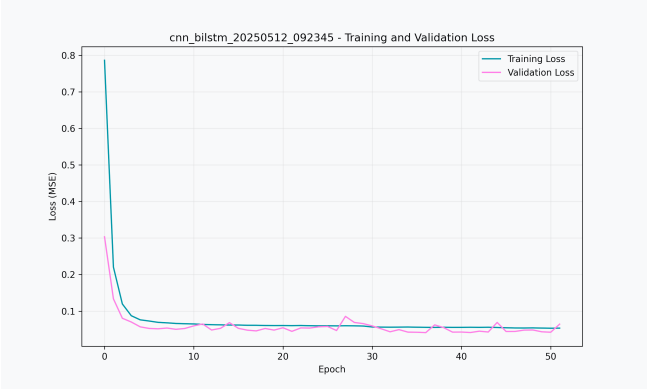

Figure 8: CNN-biLSTM model visualization by Matplotlib

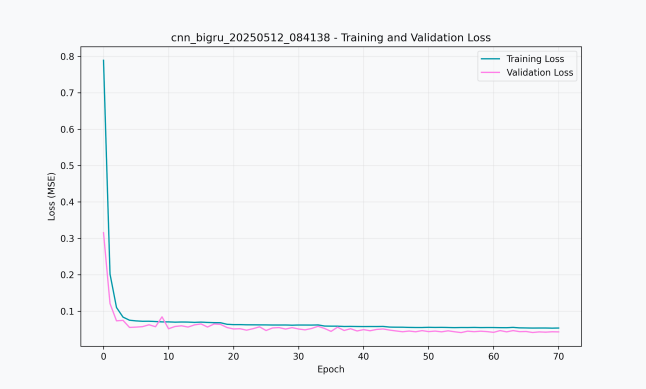

Figure 9: CNN-BiGRU model visualization by Matplotlib

When focusing solely on unseen data performance, CNN-BiGRU demonstrates slightly more stable generalization with fewer fluctuations in its validation curve. While both models achieve similar final validation loss values, CNN-BiGRU shows more consistent performance across the validation set throughout the training process. This suggests that while CNN-BiLSTM converges faster in terms of training time, CNN-BiGRU may offer marginally more reliable performance on unseen data with less variance in its predictions. However, our results challenge the theoretical computational advantage typically associated with GRUs. Despite GRUs having fewer parameters and simpler structure than LSTMs, CNN-BiLSTM trained significantly faster. This counter-intuitive result can be attributed to:

- **Backend Optimizations:** Libraries like NVIDIA's CuDNN offer highly optimized LSTM implementations that accelerate computation beyond theoretical expectations.

- **Task-Specific Learning Dynamics:** Traffic data patterns may be particularly well-suited to the CNN-BiLSTM architecture, enabling more efficient learning despite greater theoretical complexity.
- **Memory Access Patterns:** The CNN-BiLSTM implementation may benefit from more efficient memory access patterns on our specific hardware configuration.

Comparative Analysis and Model Selection

When comparing the performance of various neural network architectures for traffic prediction, it is essential to conduct a comprehensive evaluation using both qualitative observations and quantitative regression metrics. Tables 2 and 3 present the performance metrics for the five models under consideration.

Among the baseline models, BiLSTM demonstrates the strongest performance with the lowest MAE (12.6369), lowest RMSE (18.4175), and highest R^2 (0.9544). However, when examining the hybrid models, a different pattern emerges. While the CNN-BiGRU and CNN-BiLSTM exhibit slightly higher MAE values (16.8298 and 16.8748 respectively), they achieve substantially lower RMSE scores, which are 11.2363 and 11.2470 respectively, representing approximately a 39% reduction compared to the best baseline model. Additionally, both hybrid models attain superior R^2 values (0.9620 and 0.9617), indicating better overall fit to the actual traffic patterns.

Beyond the numerical metrics, the hybrid models demonstrate several important qualitative improvements:

- **Peak Capture Improvement:** Both hybrid models effectively overcome the underestimation of peak traffic volumes exhibited by the standard LSTM, and resolve the imprecise peak detection seen in GRU, which is particularly around high-congestion periods such as 08:00 and 03:30.
- **Enhanced Fluctuation Accuracy:** The integration of CNN allows both hybrid models to better recognize and follow sudden volume spikes during high-traffic intervals, directly addressing a major limitation of the standalone recurrent models.
- **Stable 24-Hour Performance:** Unlike the base models that demonstrate inconsistent results across various times of day, both CNN-BiLSTM and CNN-BiGRU deliver more uniform and accurate predictions throughout the full 24-hour traffic cycle.

Based on the comprehensive evaluation of both quantitative metrics and qualitative performance characteristics, CNN-BiGRU emerges as the optimal choice for our traffic prediction task. While it has a marginally higher MAE than the baseline BiLSTM, its substantially lower RMSE (11.2363 compared to 18.4175) indicates significantly fewer large prediction errors, which is a critical factor in traffic management applications where accurate prediction of peak congestion is essential. Furthermore, it achieves the highest R^2 value (0.9620) among all models, demonstrating superior overall predictive power. The minimal performance difference between CNN-BiGRU and CNN-BiLSTM (differences of 0.0045 in MAE, 0.0107 in RMSE, and 0.0003 in R^2) suggests that either hybrid architecture would represent a substantial improvement over the baseline models, with CNN-BiGRU offering marginally better performance across all evaluated metrics.

7 Conclusion

This project has successfully implemented a **Traffic-Based Route Guidance System (TBRGS)** that combines machine learning-based traffic prediction with intelligent route planning.

The implementation of various **deep learning models** (**LSTM**, **GRU**, and **Bi-LSTM**) has demonstrated their effectiveness in predicting traffic conditions, with each model offering different accuracy and computational efficiency advantages.

Integrating multiple **search algorithms** (**A***, **BFS**, **DFS**, **GBFS**, **UCS**, and **Fringe**) has provided a robust foundation for route finding, allowing users to choose the most appropriate algorithm for their specific needs.

The system's ability to incorporate predicted traffic conditions into route planning has resulted in more accurate and practical route recommendations. The end-to-end system combines data processing, machine learning prediction, and route finding into a cohesive solution.

The **web-based interface** provides an intuitive way for users to interact with the system, view route information, and make informed decisions about their travel plans.
Looking forward, there are several opportunities for improvement. The system could benefit from implementing ensemble methods to combine predictions from multiple models, adding support for real-time data updates, and implementing parallel processing for faster route calculations. Integrating additional data sources such as weather information and special events could further enhance prediction accuracy.

Acknowledgements

I would like to acknowledge the support of several tools that contributed to this project. ChatGPT 4.0 was helpful in drafting and refining the written content. Claude 3.7 Sonnet assisted with coding and implementation. Kaggle provided the computational resources used for training the machine learning models. Their combined support made the development process smoother and more efficient.

References

[1] Department of Transport and Planning (Victoria Government). Traffic Signal Volume Data. https://discover.data.vic.gov.au/dataset/traffic-signal-volume-data, 2025. Dataset. Published (metadata record): 11 December 2024; Last updated: 14 August 2025. Licensed under Creative Commons Attribution 4.0 International.

[2] Yoshua Bengio, Patrice Simard, and Paolo Frasconi. Learning long-term dependencies with gradient descent is difficult. *IEEE transactions on neural networks*, 5(2):157–166, 1994.

[3] Sepp Hochreiter and J"urgen Schmidhuber. Long short-term memory. *Neural computation*, 9(8):1735–1780, 1997.

[4] Kyunghyun Cho, Bart Van Merri"enboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations using rnn encoder-decoder for statistical machine translation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1724–1734, 2014.

[5] Mike Schuster and Kuldip K Paliwal. Bidirectional recurrent neural networks. *IEEE transactions on Signal Processing*, 45(11):2673–2681, 1997.

[6] GeeksforGeeks. Regression metrics, 2022. Accessed: May 13, 2025.

[7] Tianfeng Chai and Roland R Draxler. Root mean square error (rmse) or mean absolute error (mae)?– arguments against avoiding rmse in the literature. *Geoscientific model development*, 7(3):1247–1250, 2014.

[8] GeeksforGeeks. RNN vs LSTM vs GRU vs Transformers, 2023. Accessed: May 13, 2025.

[9] GeeksforGeeks. Difference between a bidirectional LSTM and an LSTM, 2023. Accessed: May 13, 2025.